

Especificación de Requisitos de Software (SRS)

Sistema: SGED — Sistema de Gestión para la Escuela Deportiva ProFútbol **Versión del documento:** 1.0 (Tercera Entrega) **Estructura:** basada en ISO/IEC/IEEE 29148:2018 **Repositorio:** https://github.com/DarwinSM21/SGED_APPWEB

Nota de redacción (resuelve OBS-01, Entrega 1A). El docente observó que los requisitos funcionales estaban redactados como títulos ("Registro de estudiantes") en vez de como requisitos. En este documento **todo requisito funcional se enuncia con la forma "El sistema deberá..."**, con un identificador único, una prioridad y un criterio de verificación comprobable.

1. Introducción

1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales de SGED, una aplicación web para la gestión administrativa y deportiva de la escuela de fútbol formativo ProFútbol. Está dirigido al equipo de desarrollo y al docente evaluador del Proyecto Fin de Curso.

1.2 Alcance

SGED cubre tres dominios:

1. **Seguridad y acceso** — personas, usuarios, roles y autenticación.

2. **Gestión académica/administrativa** — registro y mantenimiento de estudiantes y sus categorías.
3. **Dominio deportivo** — entrenadores, horarios, sesiones de entrenamiento, asistencia y evaluación diaria del desempeño.

1.3 Estado de implementación (declaración de honestidad)

Esta sección resuelve OBS-12 (Entrega 1B), donde el docente observó que el informe describía funcionalidad que no existía en el repositorio. Para evitar repetir ese error, cada requisito indica explícitamente su estado real, verificable en el código:

Estado	Significado
✓ Implementado	Existe endpoint REST funcional, con pruebas y evidencia de ejecución.
● Modelado	El esquema de base de datos existe y está migrado (Flyway), pero aún no se expone vía API REST.
□ Planificado	Solo especificado en este documento; sin esquema ni código.

Ningún requisito marcado ● o □ debe interpretarse como funcionalidad entregada.

1.4 Definiciones y acrónimos

Término	Definición
Categoría	Grupo etario de competencia (SUB-12, SUB-15, SUB-17, SUB-20).
Baja lógica	Marcar un registro como inactivo (<code>activo = FALSE</code>) sin borrarlo físicamente.
JWT	JSON Web Token (RFC 7519), credencial de sesión firmada.

JTI	Identificador único de un JWT, usado para revocarlo.
RFID	Identificación por radiofrecuencia; medio previsto para marcar asistencia.
ProblemDetail	Formato de respuesta de error de RFC 7807 / RFC 9457.
Representante	Padre, madre o tutor legal de un estudiante menor de edad.

1.5 Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements engineering.
 - ISO/IEC 25010:2011 — Modelo de calidad de producto software.
 - RFC 9110 — HTTP Semantics.
 - RFC 7519 — JSON Web Token.
 - RFC 9457 — Problem Details for HTTP APIs.
 - OWASP Top 10:2021.
-

2. Descripción general

2.1 Perspectiva del producto

SGED es un sistema cliente-servidor de tres capas:

- **Frontend:** Angular (SPA), servido por nginx con terminación TLS en :8443.
- **Backend:** API REST en Spring Boot 3.2 (Java 21), puerto :8080.
- **Persistencia:** PostgreSQL 16 (esquemas `seguridad` y `deportivo`) y Redis 7 (caché y lista de revocación de tokens).

Orquestación reproducible vía Docker Compose con imágenes fijadas por digest SHA-256.

2.2 Actores del sistema

Actor	Descripción	Rol técnico
Administrador	Gestiona usuarios, estudiantes y configuración. Único actor con permisos de escritura sobre estudiantes.	ADMINISTRADOR
Entrenador	Consulta estudiantes de sus categorías, registra asistencia y evaluación diaria.	ENTRENADOR
Usuario estándar	Consulta de solo lectura sobre estudiantes.	USER
Representante	Tutor legal del estudiante; receptor de notificaciones.	☐ No implementado

Los tres primeros roles están sembrados en `db/seed.sql` y son los que evalúan las anotaciones `@PreAuthorize` del código.

2.3 Restricciones de diseño

- **RD-01.** El sistema deberá ejecutarse íntegramente mediante contenedores Docker, sin instalación manual de dependencias en la máquina anfitriona.
- **RD-02.** Las operaciones elementales (CRUD simple, consultas paginadas) deberán resolverse con Spring Data JPA; las operaciones de agregación y actualización masiva con criterio de negocio deberán ejecutarse en el motor de base de datos mediante procedimientos almacenados versionados.
- **RD-03.** El sistema no deberá construir sentencias SQL por concatenación dinámica de cadenas en ninguna capa.

3. Requisitos funcionales

3.1 Módulo de seguridad y acceso

RF-01 — Registro de usuarios *El sistema deberá permitir que un usuario con rol ADMINISTRADOR registre nuevas cuentas de usuario, asociándolas a una persona y a uno o más roles.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** POST /api/auth/registro —
AuthController.java:63
 - **Restricción de acceso:**
@PreAuthorize("hasRole('ADMINISTRADOR')")
 - **Verificación:** un usuario no autenticado o sin rol ADMINISTRADOR deberá recibir 401/403. Prueba:
AuthServiceTest.registroExitoso,
AuthServiceTest.registroEmailDuplicado. Evidencia
OWASP A01: docs/mediciones/sec/a01-acceso-roto.txt.
-

RF-02 — Autenticación de usuarios *El sistema deberá autenticar a un usuario mediante nombre de usuario y contraseña, y deberá emitir la credencial de sesión exclusivamente en una cookie HttpOnly, Secure y SameSite=Strict, sin exponer el token en el cuerpo de la respuesta ni en almacenamiento accesible por JavaScript.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** POST /api/auth/login —
AuthController.java:99
 - **Verificación:** la respuesta deberá contener Set-Cookie con los tres atributos y no deberá contener el JWT en el cuerpo. Pruebas:
AuthServiceTest.loginConCredencialesCorrectas,
AuthServiceTest.loginConContrasenaIncorrecta.
-

RF-03 — Cierre de sesión con revocación efectiva *El sistema deberá permitir cerrar la sesión, y deberá invalidar el token emitido registrando su identificador (JTI) en una lista de revocación con tiempo de vida igual al tiempo restante del token, de modo que un token robado antes del cierre de sesión no siga siendo aceptado.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** POST /api/auth/logout —
AuthController.java:143;
RedisBlacklistService.java
 - **Verificación:** pruebas
RedisBlacklistServiceTest.revocar_guarda_el_jti_con_e
RedisBlacklistServiceTest.estaRevocado_true_si_existe_
 - **Nota:** resuelve OBS-07 y OBS-09 (Entrega 1B), donde se observó que la lista de revocación existía pero no estaba cableada a ningún endpoint.
-

RF-04 — Renovación de sesión *El sistema deberá permitir renovar una sesión vigente mediante un token de refresco, sin exigir que el usuario vuelva a introducir sus credenciales.*

- **Prioridad:** Media · **Estado:** ✓ Implementado
 - **Origen:** POST /api/auth/refresh —
AuthController.java:164
 - **Verificación:** prueba
JwtServiceTest.refresh_token_valido.
-

RF-05 — Consulta de la sesión activa *El sistema deberá permitir que el cliente consulte los datos de la sesión en curso (nombre de usuario, nombre completo y rol) a partir de la cookie de sesión, y deberá responder 401 cuando no exista sesión válida.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** GET /api/auth/me — AuthController.java:181
 - **Verificación:** con sesión válida deberá responder 200 con {username, nombre, rol}; sin sesión, 401.
-

RF-06 — Limitación de intentos de autenticación *El sistema deberá bloquear temporalmente los intentos de autenticación de un mismo usuario tras 5 fallos consecutivos dentro de una ventana de 15 minutos, y el contador no deberá reiniciarse con cada nuevo fallo dentro de esa ventana.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado

- **Origen:** `LoginAttemptService.java`; parámetros `LOGIN_MAX_INTENTOS=5`, `LOGIN_VENTANA_MINUTOS=15` (`application.yml`)
 - **Verificación:** el sexto intento deberá responder 429 con cuerpo `ProblemDetail`. Pruebas:
`LoginAttemptServiceTest.bloqueada_al_alcanzar_el_limit`
`LoginAttemptServiceTest.fallo_subsiguiente_no_reinicia`
Evidencia OWASP A07: `docs/mediciones/sec/a07-rate-limit.txt`.
-

RF-07 — Verificación de disponibilidad del servicio *El sistema deberá exponer un endpoint público de comprobación de disponibilidad que no requiera autenticación.*

- **Prioridad:** Baja · **Estado:** ✓ Implementado
 - **Origen:** `GET /api/auth/ping` — `AuthController.java:202`
 - **Verificación:** prueba `AuthServiceTest.pingRespondePong`.
-

3.2 Módulo de gestión de estudiantes

RF-08 — Listado paginado de estudiantes *El sistema deberá permitir consultar el listado de estudiantes de forma paginada, indicando en la respuesta el número de página, el tamaño, el total de elementos y el total de páginas.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** `GET /api/estudiantes` — `EstudianteController.java:27`
 - **Acceso:** ADMINISTRADOR, ENTRENADOR, USER
 - **Verificación:** pruebas
`EstudianteControllerTest.listar_devuelve_pagina`,
`EstudianteServiceTest.listar_devuelve_pagina_envuelta`.
-

RF-09 — Consulta de estudiante por identificador *El sistema deberá*

permitir consultar un estudiante por su identificador, y deberá responder 404 con cuerpo `ProblemDetail` cuando el identificador no corresponda a ningún registro.

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** GET `/api/estudiantes/{id}` — `EstudianteController.java:40`
 - **Verificación:** pruebas
`EstudianteControllerTest.buscarPorId_existente,`
`EstudianteControllerTest.buscarPorId_inexistente_da_40`
`EstudianteServiceTest.buscar_inexistente_lanza_404.`
-

RF-10 — Registro de estudiante *El sistema deberá permitir que un usuario con rol ADMINISTRADOR registre un nuevo estudiante con nombre, apellido y categoría, creando de forma transaccional el registro de persona asociado.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** POST `/api/estudiantes` — `EstudianteController.java:46`
 - **Acceso:** `@PreAuthorize("hasRole('ADMINISTRADOR')")`
 - **Verificación:** deberá responder 201 con el recurso creado.
Pruebas:
`EstudianteControllerTest.crear_devuelve_201,`
`EstudianteServiceTest.crear_persiste_persona_y_estudia`
-

RF-11 — Validación del formato de categoría *El sistema deberá rechazar toda categoría que no cumpla el patrón `SUB-NN` (donde NN es un número de una o dos cifras), y deberá responder 422 `Unprocessable Entity` con un cuerpo `ProblemDetail` que identifique el campo inválido y el motivo.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
- **Origen:** `EstudianteRequest.java` (anotación de patrón); `GlobalExceptionHandler.java`
- **Verificación:** prueba
`EstudianteControllerTest.crear_con_categoria_invalida_`
Evidencia OWASP A03: `docs/mediciones/sec/a03-`

inyeccion.txt.

RF-12 — Actualización de estudiante *El sistema deberá permitir que un usuario con rol ADMINISTRADOR actualice el nombre, apellido y categoría de un estudiante existente.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** PUT /api/estudiantes/{id} —
EstudianteController.java:54
 - **Verificación:** prueba
EstudianteControllerTest.editar_actualiza_estudiante.
-

RF-13 — Baja lógica de estudiante *El sistema deberá dar de baja a un estudiante marcándolo como inactivo, y no deberá eliminar físicamente el registro, con el fin de preservar el historial deportivo asociado.*

- **Prioridad:** Alta · **Estado:** ✓ Implementado
 - **Origen:** DELETE /api/estudiantes/{id} —
EstudianteController.java:62
 - **Verificación:** deberá responder 204 y el registro deberá permanecer en la tabla con activo = FALSE. Pruebas:
EstudianteControllerTest.eliminar_devuelve_204,
EstudianteServiceTest.eliminar_hace_baja_logica.
-

RF-14 — Conteo de estudiantes activos por categoría *El sistema deberá informar el número de estudiantes activos de una categoría, y dicho conteo deberá calcularse en el motor de base de datos mediante un procedimiento almacenado versionado, no en la capa de aplicación.*

- **Prioridad:** Media · **Estado:** ✓ Implementado
- **Origen:** GET /api/estudiantes/conteo/{categoria} —
EstudianteController.java:69;
seguridad.sp_contar_estudiantes_activos (migración V6)
- **Acceso:** ADMINISTRADOR, ENTRENADOR
- **Verificación:** pruebas
EstudianteControllerTest.contarActivos_delega_en_servi
EstudianteServiceTest.conteo_por_categoria_delega_en_f

- **Justificación:** cumple RD-02 (agregación obligatoriamente en el motor).

RF-15 — Desactivación masiva por categoría *El sistema deberá permitir dar de baja lógica, en una sola operación transaccional, a todos los estudiantes activos de una categoría, e informar el número de registros afectados; dicha operación deberá ejecutarse mediante un procedimiento almacenado versionado.*

- **Prioridad:** Media · **Estado:** ✓ Implementado
- **Origen:** POST /api/estudiantes/operaciones/desactivar-categoria —
EstudianteController.java:79;
seguridad.sp_desactivar_estudiantes_categoria
(migración v6)
- **Acceso:** @PreAuthorize("hasRole('ADMINISTRADOR')")
- **Verificación:** prueba
EstudianteControllerTest.desactivarCategoria_delega_en.

3.3 Módulo deportivo

Los requisitos de esta sección tienen su **esquema de datos migrado y versionado** (V3__dominio_deportivo.sql, V4__evaluaciones.sql), pero **aún no se exponen mediante API REST**. Se documentan aquí porque condicionan el modelo de datos entregado y el diseño de la siguiente iteración.

RF-16 — Gestión de entrenadores ● Modelado *El sistema deberá permitir registrar entrenadores asociados a una persona, con especialidad y fecha de contratación, garantizando que una misma persona no pueda registrarse dos veces como entrenador. Esquema: deportivo.entrenadores con índice único sobre id_persona.*

RF-17 — Horarios recurrentes de entrenamiento ● Modelado *El sistema deberá permitir definir horarios semanales recurrentes por*

categoría y entrenador, y deberá impedir que la hora de fin sea anterior o igual a la hora de inicio. Esquema:

`deportivo.horarios_entrenamiento`, con CHECK (`hora_fin > hora_inicio`) y CHECK (`dia_semana BETWEEN 1 AND 7`).

RF-18 — Sesiones de entrenamiento ● Modelado *El sistema deberá registrar cada sesión de entrenamiento con su fecha, categoría, entrenador responsable y estado, admitiendo únicamente los estados PROGRAMADA, EN_CURSO, FINALIZADA y CANCELADA. Esquema:* `deportivo.sesiones_entrenamiento`, con restricción CHECK sobre estado.

RF-19 — Registro de asistencia ● Modelado *El sistema deberá registrar la asistencia de cada estudiante a cada sesión, admitiendo el marcaje por RFID o manual, con estado PRESENTE, TARDE, AUSENTE o JUSTIFICADO, y deberá impedir que se registre más de una asistencia del mismo estudiante en la misma sesión. Esquema:* `deportivo.asistencias`, con UNIQUE (`id_sesion, id_estudiante`) y CHECK sobre `metodo` y `estado`.

RF-20 — Evaluación diaria del desempeño ● Modelado *El sistema deberá permitir al entrenador evaluar a cada estudiante por criterios configurables (técnica, condición física, táctica y actitud), registrando la posición jugada ese día, y deberá impedir puntajes negativos y evaluaciones duplicadas del mismo estudiante y criterio dentro de una misma evaluación. Esquema:* `deportivo.evaluaciones_diarias`, `deportivo.criterios_evaluacion`, `deportivo.detalle_evaluacion`, con CHECK (`puntaje >= 0`) y UNIQUE (`id_evaluacion, id_estudiante, id_criterio`).

RF-21 — Consulta de promedios de evaluación ● Modelado *El sistema deberá calcular el promedio de puntajes por estudiante y evaluación en el motor de base de datos. Esquema:* `vista deportivo.v_promedio_evaluacion`.

RF-22 — Notificación a representantes ☐ Planificado *El sistema deberá notificar al representante legal cuando su representado marque asistencia o registre una lesión.* Sin esquema ni implementación. Requiere resolver previamente el consentimiento del representante (ver docs/etica/ETHICS.md).

4. Requisitos no funcionales

Clasificados según las características de calidad de ISO/IEC 25010:2011. Los valores medidos provienen de la evidencia real versionada en docs/mediciones/, no de estimaciones.

4.1 Eficiencia de desempeño

RNF-01 — Tiempo de respuesta *El sistema deberá responder a las consultas paginadas de estudiantes con un percentil 95 inferior a 200 ms con caché caliente e inferior a 500 ms con caché fría, bajo una carga de 50 usuarios virtuales concurrentes.*

- **Verificación:** 3 corridas independientes de k6 (50 VUs, 30 s).
- **Resultado medido:** p95 promedio **14,18 ms** (IC 95 % $\pm$ 3,20), media 6,48 ms, throughput 404,57 RPS, **0 % de errores**. Cumple con amplio margen. Evidencia: docs/mediciones/perf/REPORT.md y los tres k6-run*.json.

RNF-02 — Caché de consultas frecuentes *El sistema deberá cachear las consultas de listado de estudiantes con un tiempo de vida de 60 segundos, y la caché no deberá corromper la deserialización de tipos temporales.*

- **Origen:** RedisCacheConfig.java; CACHE_TTL_SECONDS=60.
- **Nota:** se corrigió un defecto real por el cual la caché fallaba desde el segundo request (Jackson no leía el @class raíz) y otro por falta de soporte de java.time.Instant.

4.2 Seguridad

RNF-03 — Almacenamiento de contraseñas *El sistema no deberá almacenar contraseñas en texto plano ni de forma reversible; deberá utilizar BCrypt con factor de coste 12.* Verificación: `db/seed.sql` y `SecurityConfig.java`.

RNF-04 — Transporte cifrado *El sistema deberá ofrecer acceso mediante HTTPS con TLS 1.2 o superior.* Medido: TLS 1.3 vía nginx en `:8443`. Evidencia: `docs/mediciones/sec/a02-tls.txt` (OWASP A02).

RNF-05 — Cabeceras de seguridad *El sistema deberá enviar en todas sus respuestas las cabeceras Content-Security-Policy, X-Content-Type-Options: nosniff, X-Frame-Options: DENY y, sobre HTTPS, Strict-Transport-Security.* Evidencia: `docs/mediciones/sec/a05-cabeceras.txt` (OWASP A05).

RNF-06 — Control de acceso por rol *El sistema deberá denegar toda petición a recursos protegidos que no presente una sesión válida (401) o cuyo rol no esté autorizado para la operación (403), y dicha verificación deberá aplicarse del lado del servidor con independencia de lo que muestre la interfaz.* Evidencia: `docs/mediciones/sec/a01-acceso-roto.txt` (OWASP A01).

RNF-07 — Ausencia de SQL dinámico *El sistema no deberá construir sentencias SQL mediante concatenación de cadenas; toda consulta deberá usar parámetros vinculados o procedimientos almacenados con parámetros nombrados.* Verificación: `make audit` incluye auditoría de SQL dinámico. Evidencia: `docs/mediciones/sec/a03-inyeccion.txt` (OWASP A03).

RNF-08 — Registro de auditoría de autenticación *El sistema deberá registrar cada intento de autenticación, exitoso o fallido, incluyendo marca de tiempo, dirección IP de origen e identificador del sujeto, sin registrar nunca la contraseña.* Evidencia: `docs/mediciones/sec/a09-logging.txt` (OWASP A09).

4.3 Fiabilidad y mantenibilidad

RNF-09 — Cobertura de pruebas *El sistema deberá mantener una cobertura de instrucciones igual o superior al 60 %, verificada automáticamente en la construcción. Medido: 68 % (JaCoCo), con 34 pruebas en 7 clases. Evidencia: docs/mediciones/jacoco/.*

RNF-10 — Tipificación de errores *El sistema deberá responder los errores en formato `ProblemDetail` (RFC 9457), con `type`, `title`, `status`, `detail` e `instance`, y no deberá exponer trazas de pila ni detalles internos de implementación. Origen:*

`GlobalExceptionHandler.java`,
`ProblemDetailsAuthHandlers.java`.

RNF-11 — Versionado del esquema de datos *Todo cambio en el esquema de base de datos deberá aplicarse mediante una migración Flyway versionada e incremental; no deberá modificarse el esquema de forma manual ni automática por el ORM en tiempo de arranque. Origen: V1 a V6; ddl-auto: validate.*

4.4 Portabilidad

RNF-12 — Reproducibilidad en un solo comando *El sistema deberá levantarse completo (base de datos, caché, backend y frontend) desde una clonación limpia del repositorio mediante un único comando, en menos de dos minutos y sin configuración manual adicional más allá de copiar el archivo de variables de entorno de ejemplo. Origen: `make up`. Verificado mediante clonación real independiente en carpeta separada, no solo reiniciando el volumen local.*

RNF-13 — Fijación de dependencias de infraestructura *Las imágenes de contenedor de base de datos y caché deberán fijarse por digest SHA-256 y no por etiqueta móvil, para garantizar que dos construcciones del mismo commit usen exactamente los mismos binarios. Origen: `docker-compose.yml` (digests reales aplicados por `scripts/pin-digests.sh`).*

5. Trazabilidad

La correspondencia entre cada requisito, su implementación, su prueba automatizada y su evidencia empírica se mantiene en [docs/trazabilidad/matriz.csv](#).

El seguimiento de las observaciones emitidas por el docente en las entregas previas se mantiene en docs/observaciones/.